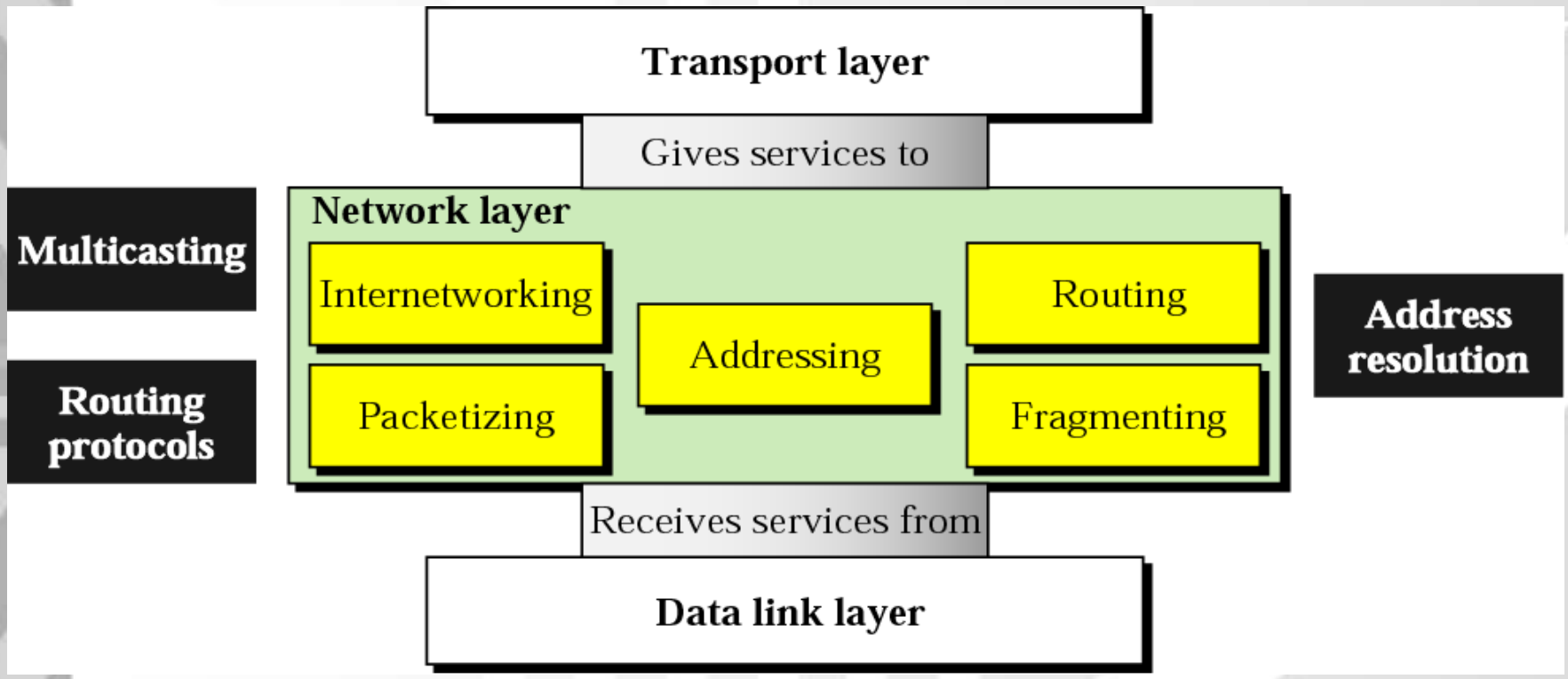
Computer Network and Communication CNS3200

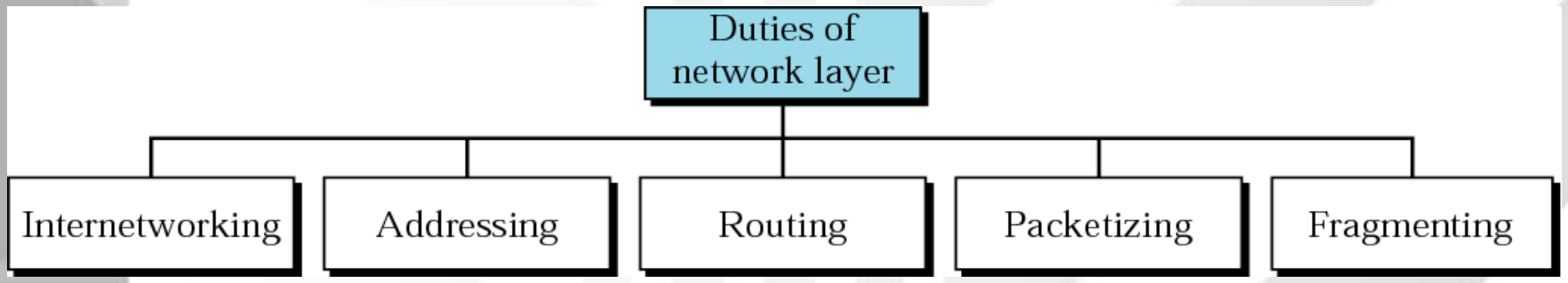
Learning Outcome

- **EXPLAIN THE FUNCTIONS OF NETWORK LAYER AND RELATED PROTOCOLS (C2)**
- **EXPLAIN THE ADDRESSING & IP CLASSESS (C4)**
- **SHOW THE CALCULATION OF THE IP ADDRESS USING THE SUBNET MASK (NS, P4)**
- **EXPLAIN ROUTING ALGORITHMS (C4)**

Functions of network layer



Network layer duties



Routing Algorithms

- To find the best route to a destination
- parameters :-
  - a. minimize mean packet delay
  - b. maximize the network throughput
  - c. minimize the number of hops along the path

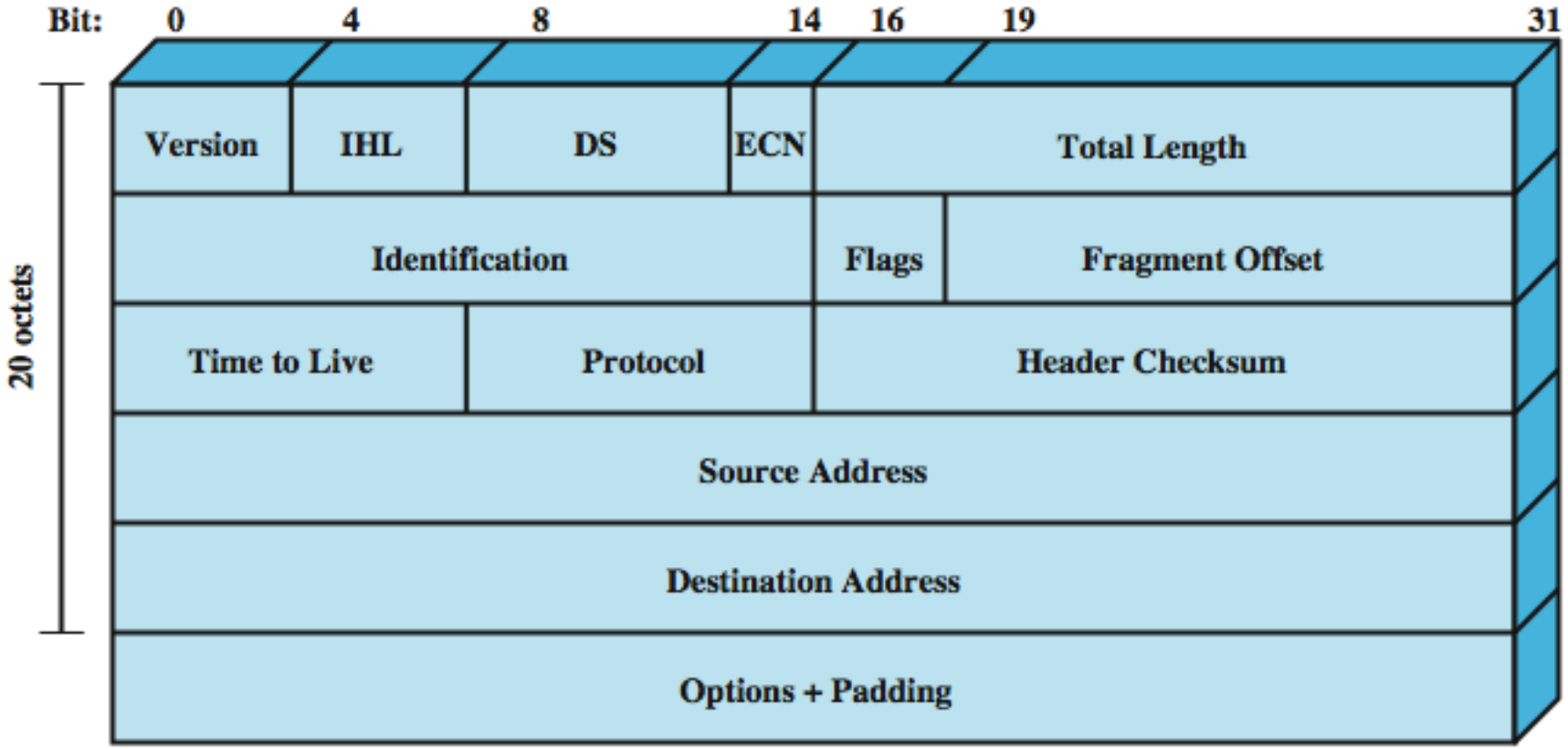
Internet Protocol (IP) v4

- IP version 4
- defined in RFC 791
- part of TCP/IP suite
- two parts
  - specification of interface with a higher layer
    - e.g. TCP
  - specification of actual protocol format and mechanisms
- will (eventually) be replaced by IPv6

IP Parameters

- source & destination addresses
- protocol
- type of Service
- identification
- don’t fragment indicator
- time to live
- data length
- option data
- user data

IPv4 Header


Bit:
0
4
8
14
16
19
31
20 octets
Version
IHL
DS
ECN
Total Length
Identification
Flags
Fragment Offset
Time to Live
Protocol
Header Checksum
Source Address
Destination Address
Options + Padding

Header Fields (1)

- Version
  - currently 4
  - IP v6 - see later
- Internet header length
  - in 32 bit words
  - including options
- DS/ECN (was type of service)
- total length
  - of datagram, in octets

Header Fields (2)

- Identification
  - sequence number
  - identify datagram uniquely with addresses / protocol
- Flags
  - More bit
  - Don’t fragment
- Fragmentation offset
- Time to live
- Protocol
  - Next higher layer to receive data field at destination

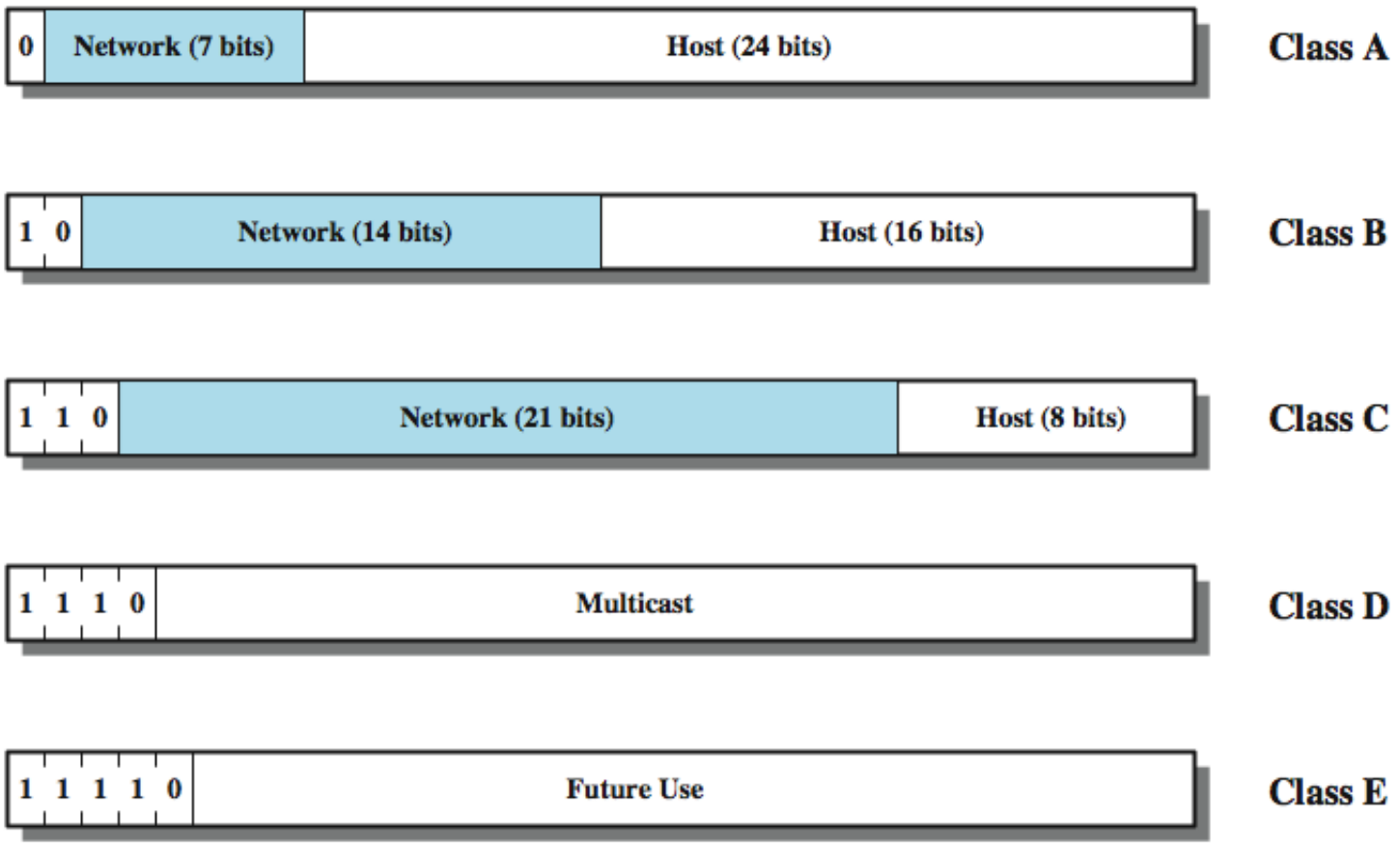
Header Fields (3)

- Header checksum
  - reverified and recomputed at each router
  - 16 bit ones complement sum of all 16 bit words in header
  - set to zero during calculation
- Source address
- Destination address
- Options
- Padding
  - to fill to multiple of 32 bits long

Data Field

- carries user data from next layer up
- integer multiple of 8 bits long (octet)
- max length of datagram (header plus data) is 65,535 octets

IPv4 Address Formats



Addressing (IPv4,IPv6)

- IPv4 Notation
  - Binary Notation
  - Dotted Decimal Notation
- IPv4 Addressing
  - Classful Addressing
  - Classless Addressing

https://www.tutorialspoint.com/classful-vs-classless-addressing

Note:

An IP address is a 32-bit address.

The IP addresses are unique and universal defines the connection to the Internet

Note:

The address space of IPv4 is 2^{32} or 4,294,967,296

IPv4 Notations

- **Binary Notation**

  - displayed in 32 bits

  - each octet is often referred to as a byte

  - Eg.

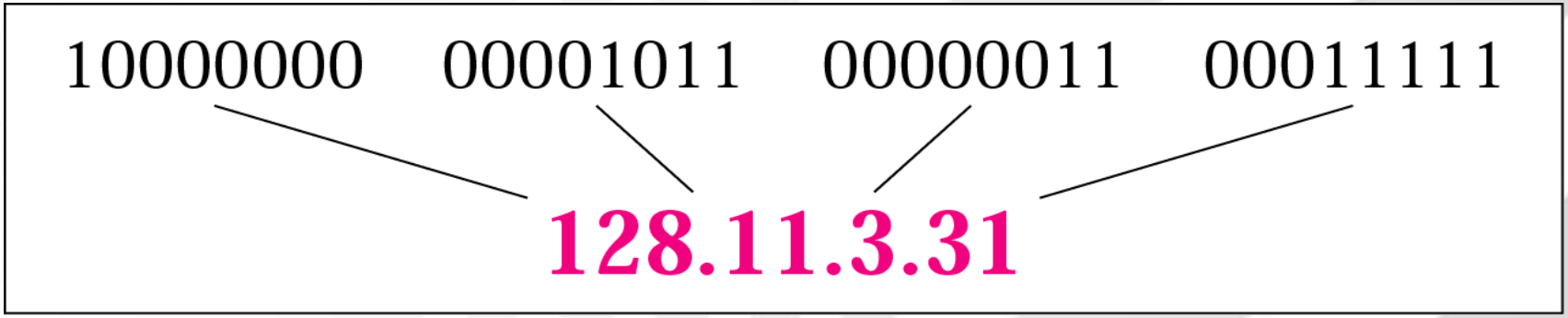
  01110101 10010101 00011101 00000010

- **Dotted-Decimal Notation**

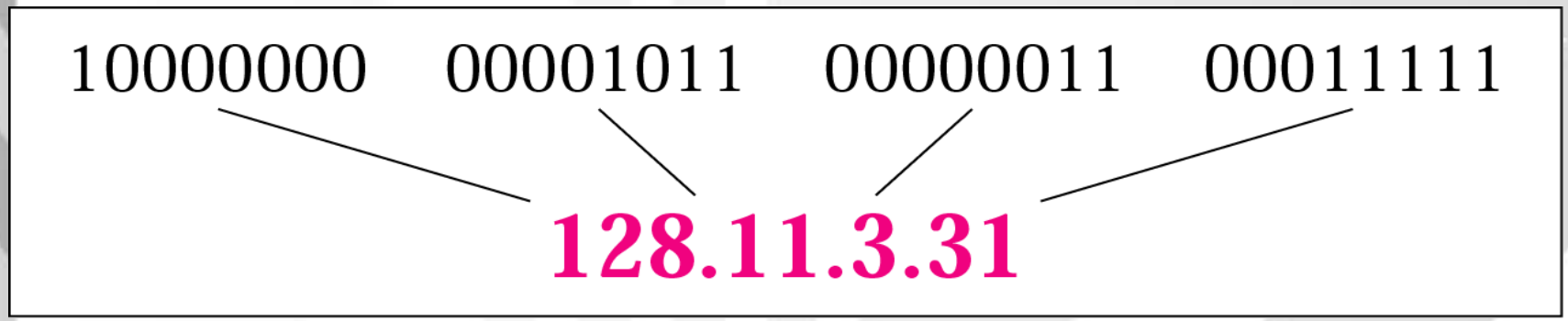
  - usually written in decimal form with a decimal point (dot) separating the byte

  - Eg. 117.149.29.2

Figure. Dotted-decimal notation



128	64	32	16	8	4	2	1



Example 1

Change the following IP addresses from binary notation to dotted-decimal notation.

a. 10000001 00001011 00001011 11101111

b. 11111001 10011011 11111011 00001111

Solution

We replace each group of 8 bits with its equivalent decimal number and add dots for separation:

a. 129.11.11.239

b. 249.155.251.15

Example 2

Change the following IP addresses from dotted-decimal notation to binary notation.

a. 111.56.45.78

b. 75.45.34.78

Solution

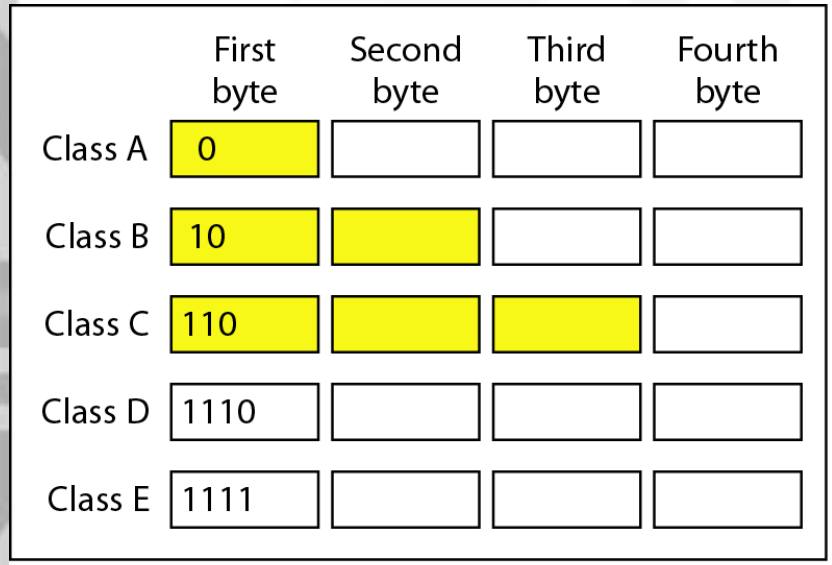
We replace each decimal number with its binary equivalent

a. 01101111 00111000 00101101 01001110
b. 01001011 00101101 00100010 01001110

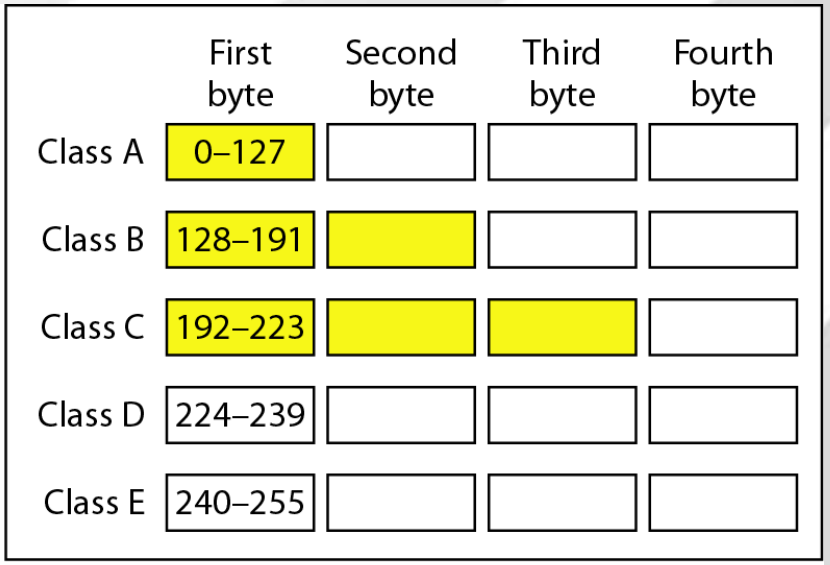
Note:

In classful addressing, the address space is divided into five classes: A, B, C, D, and E.

Finding the classes in binary and dotted-decimal notation



a. Binary notation



b. Dotted-decimal notation

Example 3

Find the class of each address.

a. 00000001 00001011 00001011 11101111

b. 11000001 10000011 00011011 11111111

c. 14.23.120.8

d. 252.5.15.111

Solution

a. The first bit is 0. This is a class A address.

b. The first 2 bits are 1; the third bit is 0. This is a class C address.

c. The first byte is 14; the class is A.

d. The first byte is 252; the class is E.

Table. ***Number of blocks and block size in classful IPv4 addressing***

Class	*Number of Blocks*	*Block Size*	*Application*
A	128	16,777,216	Unicast
B	16,384	65,536	Unicast
C	2,097,152	256	Unicast
D	1	268,435,456	Multicast
E	1	268,435,456	Reserved

Note

In classful addressing, a large part of the available addresses were wasted.

Netid and Hostid

- IP address is divided into netid and hostid
- varying lengths, depend on the class of the address.

Mask (Default Mask)

- A 32-bit address contiguous 1s and contiguous 0s
- it can be also written as /n
- n leftmost bits are 1s and (32-n) rightmost bits are 0s
- Eg. 234.56.78.69/8

  11111111 00000000 00000000 00000000

Classless Interdomain Routing

Table. ***Default masks for classful addressing***

Class	*Binary*	*Dotted-Decimal*	*CIDR*
A	11111111 00000000 00000000 00000000	255.0.0.0	/8
B	11111111 11111111 00000000 00000000	255.255.0.0	/16
C	11111111 11111111 11111111 00000000	255.255.255.0	/24

Note

Classful addressing, which is almost obsolete, is replaced with classless addressing.

Classless Addressing

- To overcome the address depletion
- No classes
- Addresses are granted in blocks (range)
- Size of block varies based on the nature & size of entity

Restriction

To simplify the handling addresses, the Internet authorities impose three restrictions:

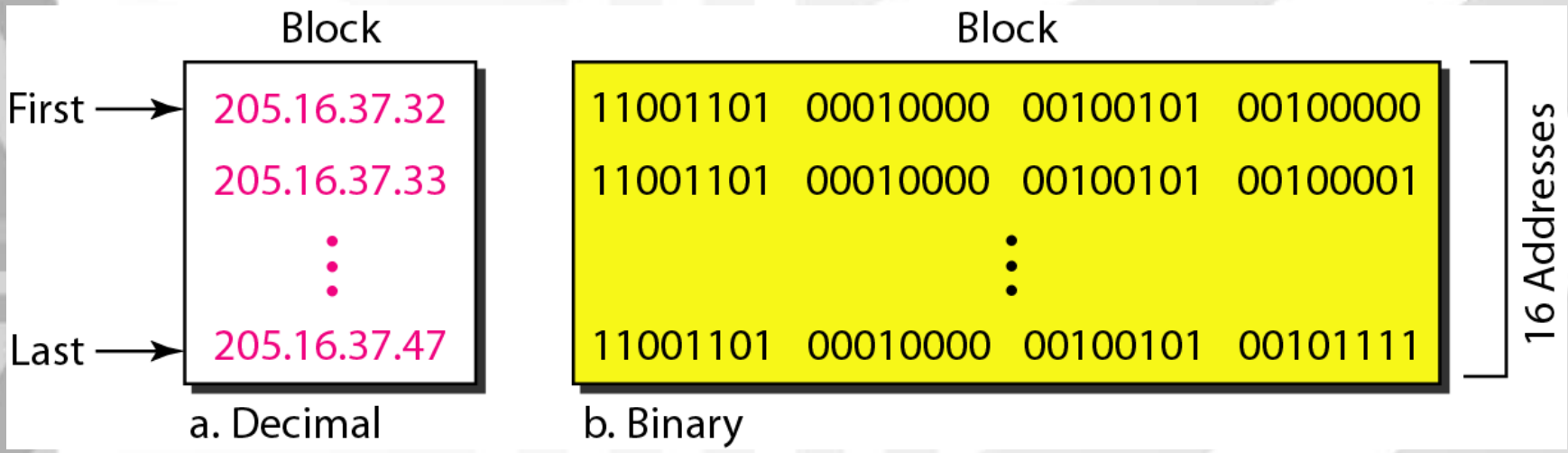
1. The addresses in a block must be contiguous
2. The number of addresses in a block must be a power of 2 (1, 2, 4, 8, 16 …)
3. The first address must be evenly divisible by the number of addresses

Example

The next figure shows a block of addresses, in both binary and dotted-decimal notation, granted to a small business that needs 16 addresses.

We can see that the restrictions are applied to this block. The addresses are contiguous. The number of addresses is a power of 2 ($16 = 2^4$), and the first address is divisible by 16. The first address, when converted to a decimal number, is 3,440,387,360, which when divided by 16 results in 215,024,210.

Figure. *A block of 16 addresses granted to a small organization*



Note

In IPv4 addressing, a block of addresses can be defined as x.y.z.t */n* in which x.y.z.t defines one of the addresses and the */n* defines the mask.

Note

The first address in the block can be found by setting the rightmost $32 - n$ bits to 0s.

Example 4

A block of addresses is granted to a small organization. We know that one of the addresses is 205.16.37.39/28. What is the first address in the block?

Solution

The binary representation of the given address is

11001101 00010000 00100101 00100111

If we set 32−28 rightmost bits to 0, we get

11001101 00010000 00100101 0010000

or

205.16.37.32.

This is actually the block shown in Figure slide 24

Note

The last address in the block can be found by setting the rightmost 32 − n bits to 1s.

Example 5

Find the last address for the block in Example 4

Solution

The binary representation of the given address is

If we set 32 − 28 rightmost bits to 1, we get

or

This is actually the block shown in Figure slide 24

Note

The number of addresses in the block can be found by using the formula 2^{32-n}.

Example 6

Find the number of addresses in Example 4

Solution

The value of n is 28, which means that Number of addresses is 2^{32-28} or 16.

Exercise

1) *In a block of addresses, we know the IP address of one host is 25.34.12.56/16. What are the first address (network address) and the last address (limited broadcast address) in this block?*

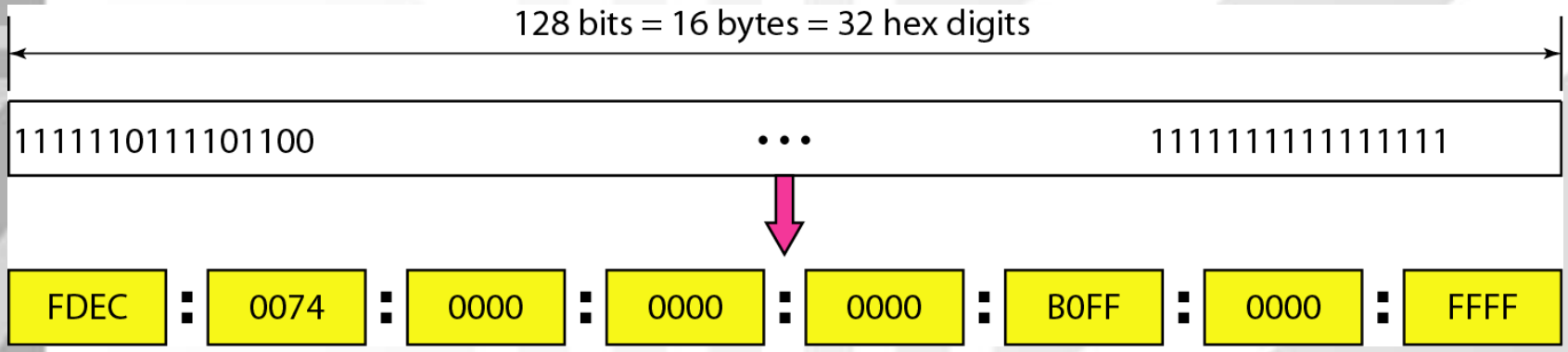
1) *Write the following masks in slash notation (/n)*
   a. 255.255.255.0
   b. 255.0.0.0
   c. 255.255.224.0
   d. 255.255.240.0

3) *Find the range of addresses in the following blocks*
   a. 123.56.77.32/29 *b. 17.34.16.0/23*

IPv6

Despite all short-term solutions, address depletion is still a long-term problem for the Internet. This and other problems in the IP protocol itself have been the motivation for IPv6.

Figure 19.14 *IPv6 address in binary and hexadecimal colon notation*

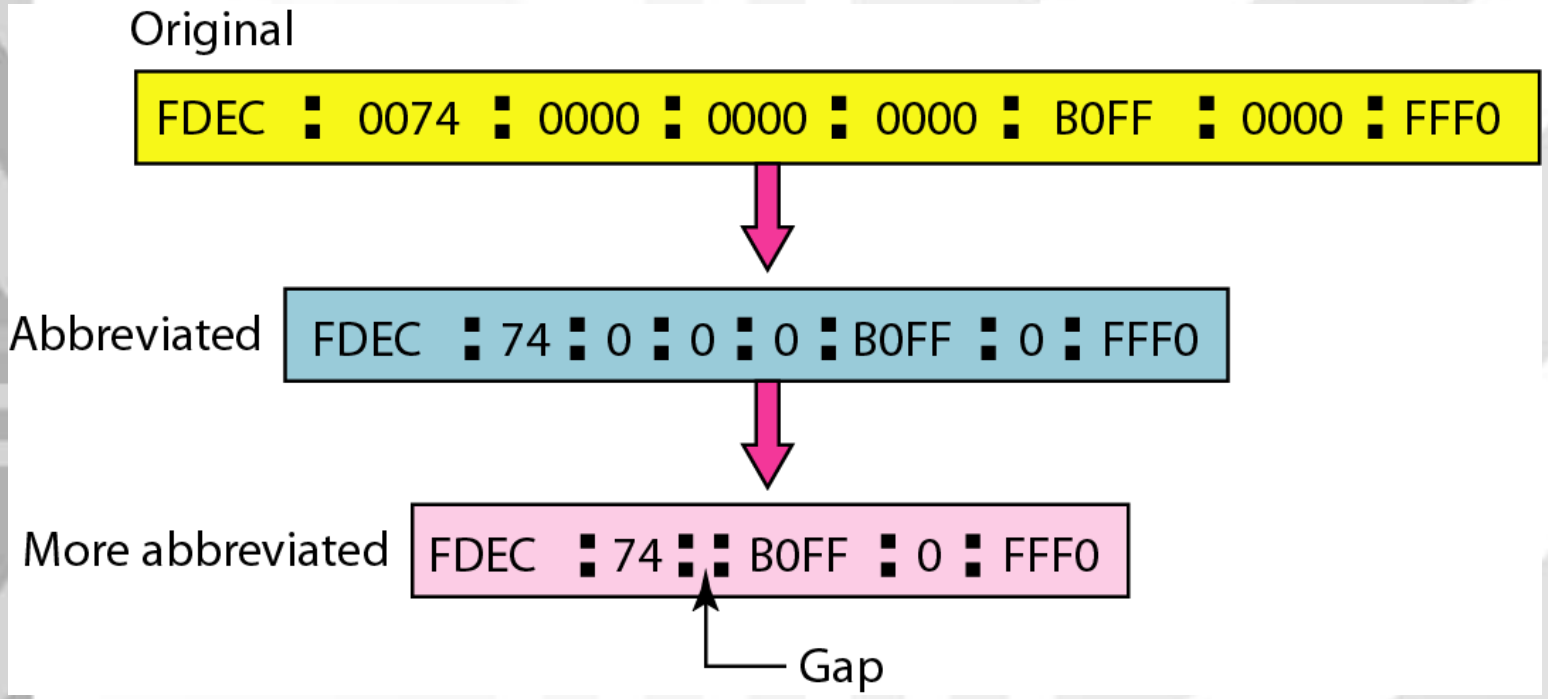


An IPv6 address is 128 bits long.

Why Change IP?

- Address space exhaustion
  - two level addressing (network and host) wastes space
  - network addresses used even if not connected
  - growth of networks and the Internet
  - extended use of TCP/IP
  - single address per host
- requirements for new types of service

Figure 19.15 *Abbreviated IPv6 addresses*



Expand the address 0:15::1:12:1213 to its original.

Solution

We first need to align the left side of the double colon to the left of the original pattern and the right side of the double colon to the right of the original pattern to find how many 0s we need to replace the double colon.

```
xxxx:xxxx:xxxx:xxxx:xxxx:xxxx:xxxx:xxxx
  0:   15:                   :    1:   12:1213
```

This means that the original address is.

```
0000:0015:0000:0000:0000:0001:0012:1213
```

Routing

Routing Algorithms

- 2 common methods of Adaptive Routing Methods:
  - Distance Vector Routing (Bellman-Ford Algorithm)
    - Use only knowledge of attached links and neighbors
  - Link State Routing (Dijkstra's Algorithm)
    - Use global knowledge about topology and cost

Distance Vector (DV) Routing

- Basic idea: each network node maintains a **Distance Vector table** containing the ***distance*** between itself and ALL possible destination nodes.

Information kept by DV router

1. Each router has an ID

2. Associated with each link connected to a router, there is a link cost (static or dynamic)

The metric issue!

Distances are based on a chosen metric**:**

– Metric: *usually hops or delay*

Distance Vector Routing

- The DV calculation is based on minimizing the cost to each destination.

Distance Vector Table Initialization

Distance to itself = 0
Distance to ALL other routers = infinity number

Distance Vector Routing

1. Router transmits its *distance vector* to all of its neighbors.

2. Each router receives and saves the most recently received *distance vector*.

Distance Vector Routing

3. A router **recalculates** its distance vector when:
   - It receives a *distance vector* from a neighbor containing different information.
   - It discovers that a link to a neighbor has gone down (i.e., a topology change).
4. Each router uses complete information on the network topology to compute the *shortest path route.*

Distance Vector Algorithm

Basic idea:

- Each node periodically sends its own distance vector estimate to neighbors
- When a node x receives new DV estimate from neighbor, it updates its own DV.

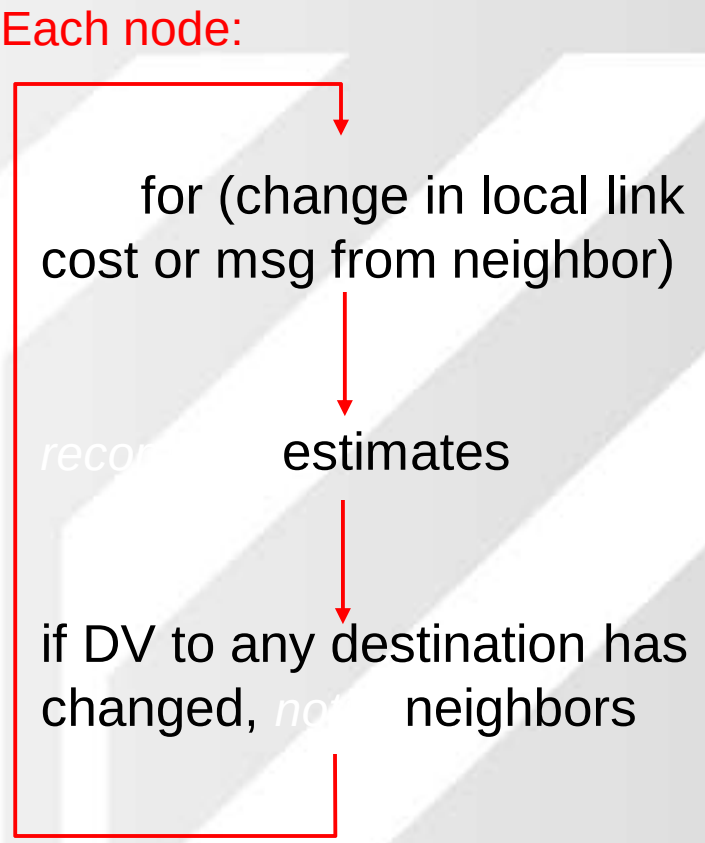
Distance Vector Algorithm

Iterative, asynchronous:
each local iteration caused by:

- local link cost change
- DV update message from neighbor

Distributed:

- each node notifies neighbors *only* when its DV changes
  - neighbors then notify their neighbors if necessary



$D_x(y) = \min\{ c(x,y) + D_y(y),\ c(x,z) + D_z(y) \}$
$= \min\{ \quad 2+0 \quad , \quad 7+1 \quad \} = \mathbf{2}$

$D_x(z) = \min\{ c(x,y) + D_y(z),\ c(x,z) + D_z(z) \}$

$= \min\{2+1 , 7+0\} = \mathbf{3}$

node x table

cost to

from	x	y	z
x	0	2	7
y	∞	∞	∞
z	∞	∞	∞

cost to

from	x	y	z
x	0	2	3
y	2	0	1
z	7	1	0

node y table

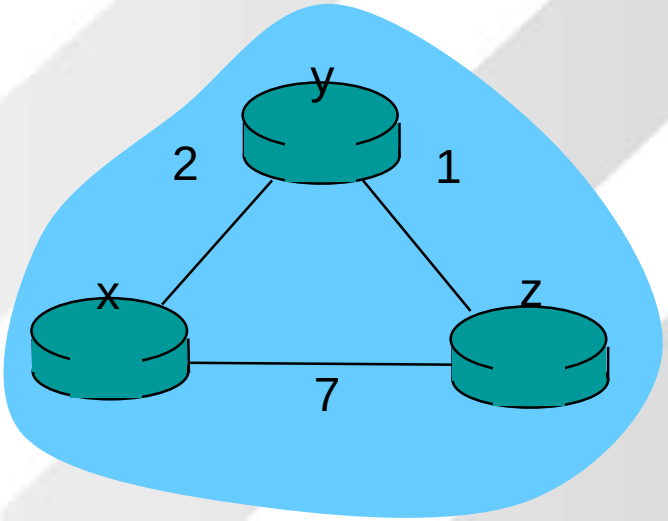
cost to

from	x	y	z
x	∞	∞	∞
y	2	0	1
z	∞	∞	∞

node z table

cost to

from	x	y	z
x	∞	∞	∞
y	∞	∞	∞
z	7	1	0



time

node x table

from \ cost to	x	y	z
x			
y			
z			

node y table

from \ cost to	x	y	z
x			
y			
z			

node z table

from \ cost to	x	y	z
x			
y			
z			

node x table

from \ cost to	x	y	z
x			
y			
z			

node y table

from \ cost to	x	y	z
x			
y			
z			

node z table

from \ cost to	x	y	z
x			
y			
z			

node x table

from \ cost to	x	y	z
x			
y			
z			

node y table

from \ cost to	x	y	z
x			
y			
z			

node z table

from \ cost to	x	y	z
x			
y			
z			

node x table

from	cost to x	cost to y	cost to z
x	0	2	7
y	∞	∞	∞
z	∞	∞	∞

from	cost to x	cost to y	cost to z
x	0	2	3
y	2	0	1
z	7	1	0

from	cost to x	cost to y	cost to z
x	0	2	3
y	2	0	1
z	3	1	0

node y table

from	cost to x	cost to y	cost to z
x	∞	∞	∞
y	2	0	1
z	∞	∞	∞

from	cost to x	cost to y	cost to z
x	0	2	7
y	2	0	1
z	7	1	0

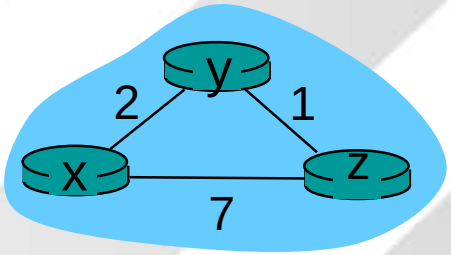
from	cost to x	cost to y	cost to z
x	0	2	3
y	2	0	1
z	3	1	0

node z table

from	cost to x	cost to y	cost to z
x	∞	∞	∞
y	∞	∞	∞
z	7	1	0

from	cost to x	cost to y	cost to z
x	0	2	7
y	2	0	1
z	3	1	0

from	cost to x	cost to y	cost to z
x	0	2	3
y	2	0	1
z	3	1	0



time

Link State Routing

- Each node maintains the full graph by collecting the updates from all other nodes
- The set of all links forms the complete graph
- Routing is performed using shortest computations on the graph

A Link-State Routing Algorithm

Dijkstra's algorithm

- net topology, link costs known to all nodes
  - accomplished via "link state broadcast"
  - all nodes have same info
- computes least cost paths from one node ('source") to all other nodes
  - gives forwarding table for that node
- iterative: after k iterations, know least cost path

Notation:

- c(x,y): link cost from node x to y; = ∞ if not direct neighbors
- D(v): current value of cost of path from source to dest. V
- p(v): predecessor node
- N': subset of nodes whose least cost path definitively known

Figure. ***Link state knowledge***

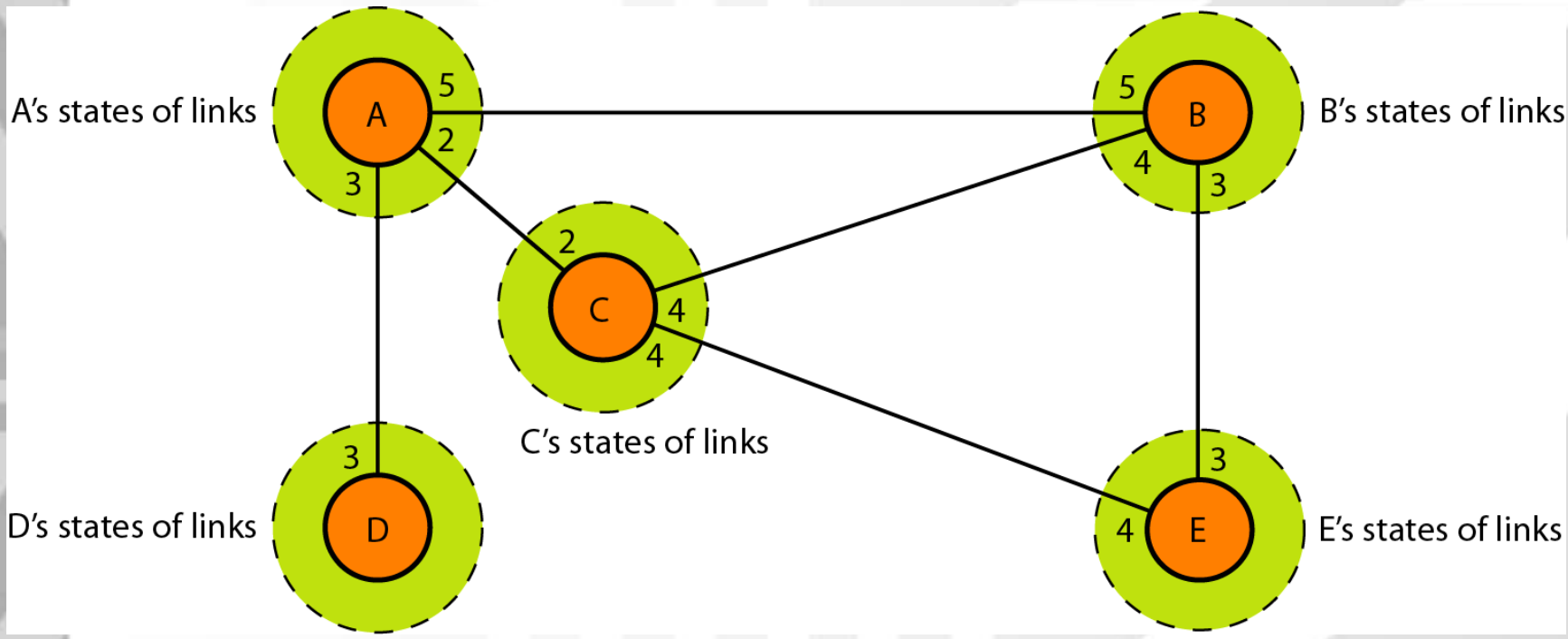
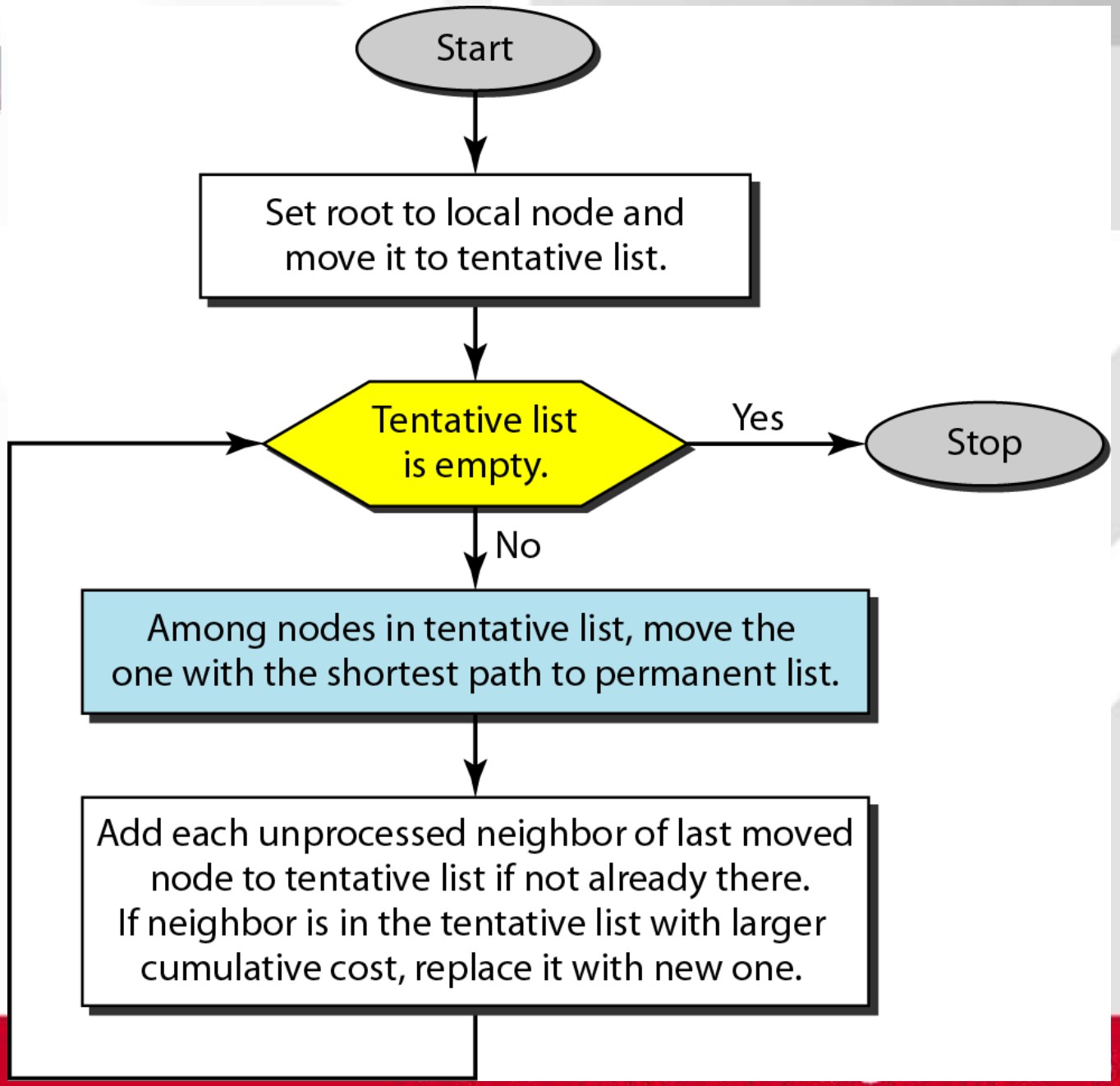


Figure. ***Dijkstra algorithm***



Permanent and Tentative List

- **Permanent**
- 1. Empty
- 2. A(0)
- 3.A(0),C(2)
- 4. A(0),C(2),D(3)
- 5. A(0),C(2),D(3),B(5)
- 6.A(0),C(2),D(3),B(5),E(6 :C)

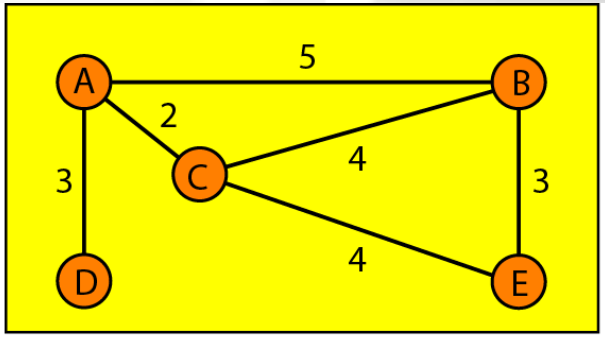
- **Tentative**
- 1. A(0)
- 2.B(5),C(2),D(3)
- 3.D(3),B(5),E(6:C)
- 4. B(5),E(6 :C)
- 5. E(6 :C)
- 6. Empty

Permanent and Tentative List

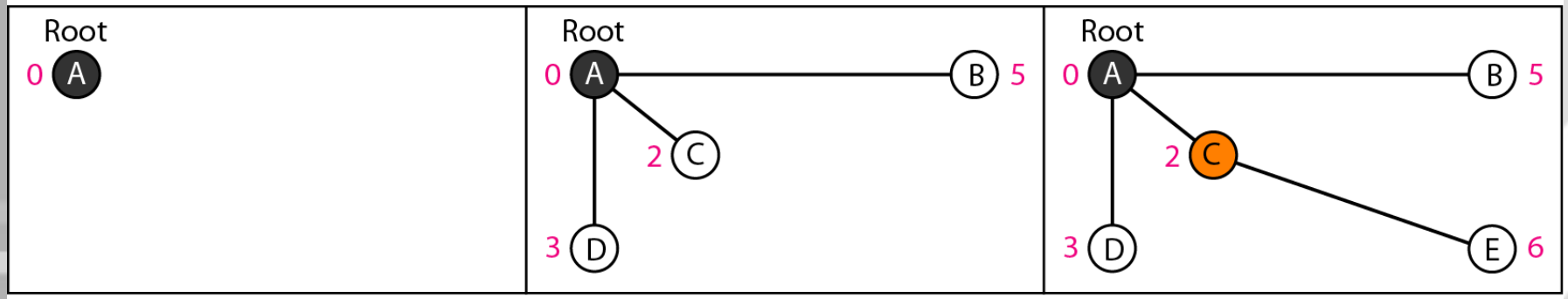
- **Permanent**
- 1. Empty
- 2.
- 3.
- 4.
- 5.
- 6.

- **Tentative**
- 1. A(0)
- 2.
- 3.
- 4.
- 5.
- 6. Empty

Figure *Example of formation of shortest path tree*



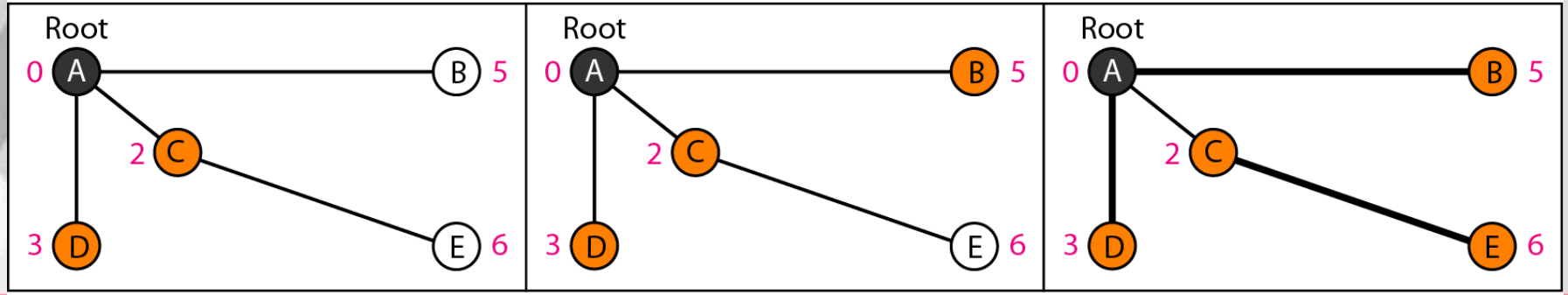
Topology



1. Set root to A and move A to tentative list.

2. Move A to permanent list and add B, C, and D to tentative list.

3. Move C to permanent and add E to tentative list.



4. Move D to permanent list.

5. Move B to permanent list.

6. Move E to permanent list (tentative list is empty).

Table . ***Routing table for node A***

Node	*Cost*	*Next Router*
A	0	—
B	5	—
C	2	—
D	3	—
E	6	C